

Testing the Tests — Opportunities for Corrections and Improvements in NIST SP 800-22r1a and its Reference Code

Elias Riesinger
elias.riesinger@proton.me

Jürgen Fuß
juergen.fuss@fh-hagenberg.at

University of Applied Sciences Upper Austria

Abstract

During an analysis of the NIST SP 800-22r1a document, which provides a test suite to validate random number generators and their reference implementation, various issues were identified, including imprecise probability constants, erroneous example calculations, and discrepancies within test descriptions. Here, we provide a technical analysis of the Statistical Test Suite, documenting inconsistencies and deficiencies in both the theoretical specification and the official C reference implementation. The analysis also reveals concrete implementation bugs and structural limitations in the reference codebase. Rather than revising any of the statistical tests, this work documents these flaws to support the ongoing revision process of the standard and to encourage more reliable and maintainable implementations.

1 Introduction

In this article, we summarize the findings of an in-depth analysis of the Statistical Test Suite specified by NIST SP 800-22r1a [3] and its reference implementation [5]. During the analysis, we discovered inconsistencies as well as errors in the description of the tests and opportunities for improvement in the reference imple-

mentation, which are covered in Sections 2 and 3, respectively.

We do not attempt to rededuce or refute the mathematical foundations of the tests. Instead, our goal is to document inconsistencies, technical issues, and other errors so that they can be addressed in the ongoing revision process of SP 800-22.

2 Flaws in the Specification

This section highlights the flaws found in SP 800-22r1a. Unless otherwise noted, the issues discussed also apply to the reference implementation. For a description of the tests we refer to SP 800-22r1a [3].

2.1 Test for the Longest Run of Ones in a Block (Sec. 2.4 and 3.4)

The test specification includes a table listing the number of blocks N for different block lengths M in Section 2.4.4 [3, 2-9]. The test description also specifies the minimum length N of a sequence to be used for each given block length M [3, 2-8]. These values are valid only for bitstreams matching exactly the minimum required length, as per the specification. With longer sequences, parts would be left untested. A comparison with the reference

implementation shows that the given values N are not actually used. Instead, the number of blocks is dynamically computed for each sequence tested [5, `src/longestRunOfOnes.c`, l. 59].

Furthermore, the constants $\pi_0, \pi_1, \dots$ for each block length are specified only to four decimal places in Section 3.4 [3, 3-4, 3-5]. For example, a 749,999 bit long bitstream would consist of $N = 749999/128 \approx 5859$ blocks. The block count would be multiplied by the probability constants. Multiplying with block counts N this large leads to a noticeable loss of precision. For accurate test results, the probabilities should be recalculated.

2.2 Binary Matrix Rank Test (Sec. 2.5 and 3.5)

The constants p_M, p_{M-1} , and p_{M-2} are given with only four decimal places [3, 2-11, 3-5], which is too imprecise for actual use.

The first example calculation in Section 2.5.4 contains a typo: $e^{0.596953/2} = 0.741948$ is incorrect [3, 2-11]; the exponent lacks the minus sign, the correct computation is $e^{-0.596953/2}$. However, the given result is correct.

2.3 Non-overlapping Template Matching Test (Sec. 2.7 and 3.7)

Although the specification claims that the implementation supports template lengths m from 2 to 10 [3, 2-16], it actually supports lengths up to 21, as indicated by the contents of the `template` directory [5]. Furthermore, the second example omits the PRNG parameters used to generate the input [3, 2-16], making it impossible to replicate.

2.4 Overlapping Template Matching Test (Sec. 2.8 and 3.8)

The specification does not specify the type of templates used in this test [3, 2-17], with only an indirect mention in the mathematical background section [3, 3-12].

Only a review of the relevant implementation source code, lines 34–35 of the code file `src/overlappingTemplateMatchings.c` [5], clarifies the fact that the templates consist only of bits with value '1'.

Also missing is an explanation of how the probability constants are calculated. The mathematical background section [3, 3-12] only shows an approximation method. The actual calculation method used was developed by Hamano and Kaneko [1] as a correction to a previous version of the test. Although this method was used to calculate the constants, it is not explained in SP 800-22, but only referenced.

The results of the second calculation example [3, 2-19] are incorrect. A manual recalculation, starting from the intermediate results $v_0, v_1, \dots, v_5$, shows that the subsequent result $\chi^2(obs)$ is incorrect. However, the value given for $\chi^2(obs)$ matches the output of the reference implementation, which is known to be incorrect [2]. For further information on the Overlapping Template Matching Test bug, see Section 3.1.

2.5 Maurer's "Universal Statistical" Test (Sec. 2.9 and 3.9)

The second example is not reproducible due to missing PRNG parameters [3, 2-23].

2.6 Linear Complexity Test (Sec. 2.10 and 3.10)

The results and intermediate values of the second example [3, 2-26], match those of the reference implementation, which contains a bug, as

shown in Section 3.2. A manual recalculation of the example shows that the intermediate result for $\chi^2(obs)$ is wrong, while the preceding result for $v_0, v_1, \dots, v_6$ is still correct.

Furthermore, probability constants are only specified as approximations. A recalculation, using the formulas given in Section 3 [3, 3-16, 3-17], yields the following values: $\pi_0 = \frac{1}{96}$, $\pi_1 = \frac{1}{32}$, $\pi_2 = \frac{1}{8}$, $\pi_3 = \frac{1}{2}$, $\pi_4 = \frac{1}{4}$, $\pi_5 = \frac{1}{16}$, $\pi_6 = \frac{1}{48}$. It is unclear why NIST opted for decimal approximations instead.

2.7 Serial Test (Sec. 2.11 and 3.11)

In the description of the test (Section 2.11), either the description of Step 5 or the example calculation are incorrect.

The test description provides the following formulas [3, 2-28]:

$$P\text{-value}1 = \mathbf{igamc}(2^{m-2}, \nabla\psi_m^2) \text{ and} \quad (1)$$

$$P\text{-value}2 = \mathbf{igamc}(2^{m-3}, \nabla^2\psi_m^2). \quad (2)$$

The example calculation provides $m = 3$ and calculates $\nabla\psi_m^2 = 1.6$, $\nabla^2\psi_m^2 = 0.8$ [3, 2-27]. Based on the example, the formulas would be

$$P\text{-value}1 = \mathbf{igamc}(2^{m-2}, \frac{\nabla\psi_m^2}{2}) \text{ and} \quad (3)$$

$$P\text{-value}2 = \mathbf{igamc}(2^{m-3}, \frac{\nabla^2\psi_m^2}{2}). \quad (4)$$

Additionally, the resulting P-values in the example are wrong for both the formulas (1) and (2) given in the description and the alternative ones (3) and 4.

The results calculated using our alternative formulas can be found in Section 2.11.6 of SP 800-22r1a [3]. The formulas are also given in the mathematical background section [3, 3-18] and the reference implementation uses them [5, `src/serial.c`, l. 20–21]. This indicates that the formulas (1) and (2) given in the test description are incorrect. The origin of the incorrect example calculation results is unclear.

2.8 Random Excursions Test (Sec. 2.14 and 3.14)

The necessary probability values are rounded to only four decimal places [3, 3-24], making a recalculation necessary. This choice is puzzling, as all probabilities can be expressed as exact fractions.

3 Flaws in the Implementation

This section outlines defects found in the reference implementation, in addition to the issues of the specification.

3.1 Overlapping Template Matching Test Bug

Identified in 2022 [2], the problem here is that the calculation of the probability constants was not corrected according to Hamano and Kaneko [1], even though the specification was updated according to their proposal. A workaround for this bug is to delete lines 40–44 of `src/overlappingTemplateMatchings.c`, but this is only valid for input sizes of around 10^6 bits. A proper fix requires re-implementing the probability calculation, as already done by Kowalska et al. [2].

3.2 Linear Complexity Test Bug

The reference implementation uses a wrong probability constant; in line 13 of the file `src/linearComplexity.c` [5], π_0 is listed as 0.01047 instead of 0.010417, rendering all results subtly wrong. This, again, raises the question why the exact fractions (see Section 2.6) were not used.

4 Structural Limitations and Benefits of a Custom Implementation

In addition to the concrete flaws identified earlier, the reference implementation code exhibits further structural deficiencies. These do not affect the correctness of the test results, but they hinder usability, extensibility, and long-term maintainability.

A notable issue is the lack of documentation. Several test modules (e.g. `src/frequency.c`, `src/runs.c` [5]) have neither in-line comments nor other code documentation. Understanding the internal logic of these functions often requires a detailed comparison with the specification in SP 800-22, complicating modifications or extensions.

The architectural design of the reference implementation is outdated. It can only be used as the command-line tool `assess` and lacks the modularity required for integration into other software systems. The tool was designed for interactive use only [3, 5-6, 5-7]. It was not designed to be used as a library or invoked from external tools. This limits its usefulness in automated testing workflows and reproducible research settings.

To address these and other issues, we developed a custom implementation in *Rust*, available at <https://github.com/riesinger-e/nist-sts>. While also providing a command-line tool (which was developed to allow non-interactive use), it is designed as a reusable library. All tests can be executed programmatically or individually, enabling integration into larger toolchains or iterative testing setups. For maximum compatibility, we provide a programming interface in *C*, while also providing *Python* bindings to cater to scientific users.

Performance is another area with room for improvement, as also noted in prior work (e.g. [4]). The current reference implementa-

tion stores each individual bit as a full byte [5, `include/defs.h`, l. 50], resulting in an eight-fold increase in memory usage compared to what is strictly necessary. This approach leads to inefficient bit-level processing in test routines, which affects runtime performance. Previous publications [4] have demonstrated that more compact bit storage can lead to improved performance. We could verify this with our custom implementation, which achieves execution times under 1 % of those measured for the reference implementation for certain tests.

5 Conclusion

Overall, the reference implementation of NIST SP 800-22r1a shows signs of aging, and its version control practices appear insufficient—evident in the bug in the Overlapping Template Matching Test. Some of the issues of the specification, such as example calculations that are no longer reproducible, may have resulted from poorly coordinated updates.

Most of the identified problems are minor oversights that could have been detected through rigorous review prior to publication. We hope that the ongoing revision process will address these shortcomings, resulting in a more accurate, maintainable, and modernized statistical test suite.

References

- [1] Kenji Hamano and Toshinobu Kaneko. “Correction of Overlapping Template Matching Test Included in NIST Randomness Test Suite”. *IEICE Transactions* 90-A (Sept. 2007), pp. 1788–1792. DOI: 10.1093/ietfec/e90-a.9.1788 (cit. on pp. 2, 3).
- [2] Katarzyna Anna Kowalska, Davide Fogliano, and Jose Garcia Coello. “On the revision of NIST 800-22 Test

- Suites”. Cryptology ePrint Archive, Paper 2022/540. 2022. URL: <https://eprint.iacr.org/2022/540> (visited on 09/22/2024). preprint (cit. on pp. 2, 3).
- [3] Andrew Rukhin et al. *A statistical test suite for random and pseudorandom number generators for cryptographic applications*. 2010. DOI: 10.6028/nist.sp.800-22r1a (cit. on pp. 1–4).
- [4] A. Suciu et al. “Byte-oriented efficient implementation of the NIST statistical test suite”. In: *2010 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR)*. Vol. 2. 2010, pp. 1–6. DOI: 10.1109/AQTR.2010.5520837 (cit. on p. 4).
- [5] *NIST Statistical Test Suite - Reference Implementation*. URL: <https://csrc.nist.gov/projects/random-bit-generation/documentation-and-software> (cit. on pp. 1–4).